

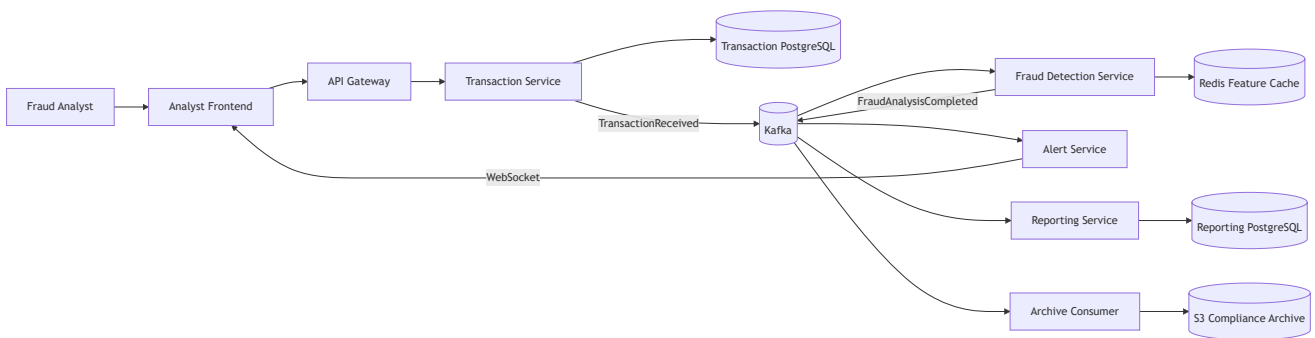
Real-Time Financial Fraud Detection Architecture

1. Overview

The bank needs to replace its batch fraud system with a real-time system. The solution should analyse up to 1,000 transactions per second, keep transactions replayable for compliance, scale during busy periods, and show high-risk transactions to fraud analysts.

The proposed design uses microservices and Kafka. Transactions are accepted through an API, published as events, analysed by a machine learning service, and then sent to alerting and reporting services.

2. Architecture



The services run in containers and can be scaled separately. Kafka keeps the events for replay and allows a service to continue from where it stopped. S3 provides longer-term storage for compliance.

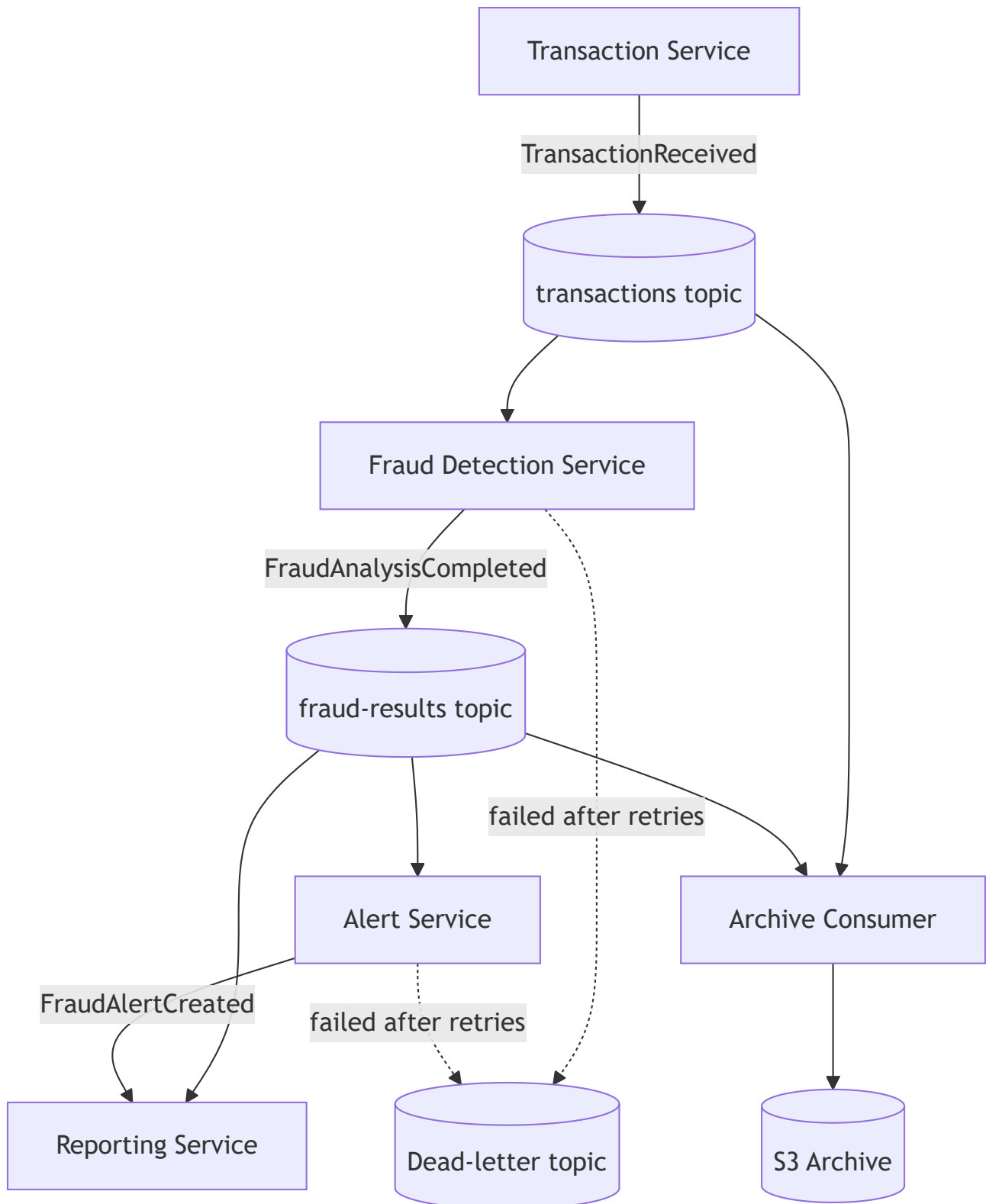
3. Main services

Service	Responsibility	Input and output
API Gateway	Authentication, routing, and rate limiting	REST/WebSocket requests and responses
Transaction Service	Validate transactions, assign IDs, save the transaction, and publish an event	Transaction request → TransactionReceived
Fraud Detection Service	Read features and run the fraud model	TransactionReceived → FraudAnalysisCompleted
Alert Service	Identify high-risk results and notify analysts	Fraud result → alert and WebSocket message
Reporting Service	Save results and support analyst searches	Fraud result → reporting database
Archive Consumer	Save events for compliance and replay	Kafka events → S3 archive

The main communication method is event-based through Kafka. This keeps the services loosely coupled. REST is used for submitting transactions and for frontend queries, because those requests need a direct

response. Fraud analysis is asynchronous so the API does not have to wait for the machine learning model.

4. Event flow



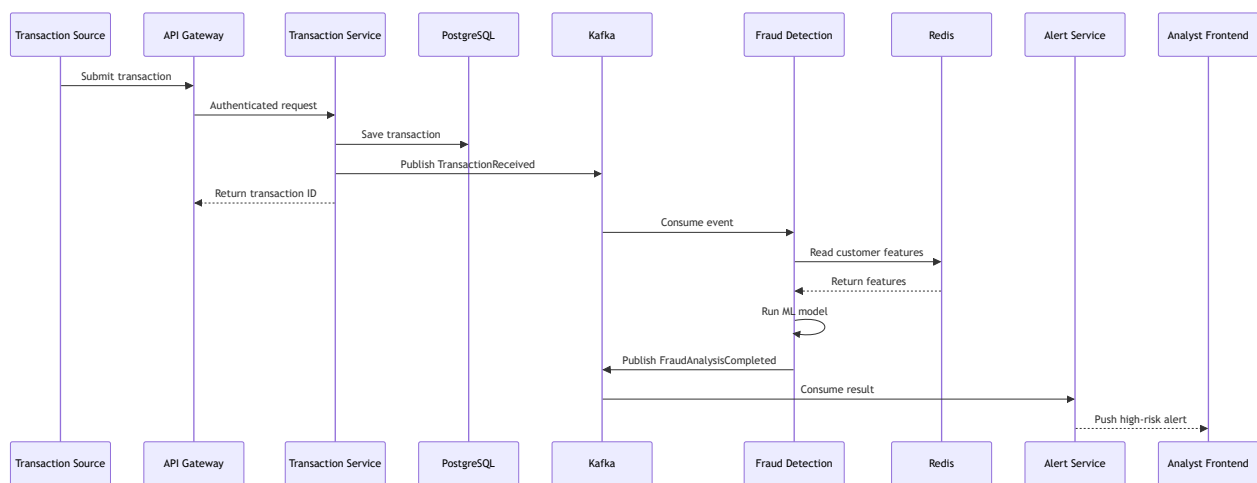
Important events are:

- **TransactionReceived** : published by the Transaction Service when a transaction has been validated and stored.
- **FraudAnalysisCompleted** : published by the Fraud Detection Service with the risk score, decision, and model version.
- **FraudAlertCreated** : published by the Alert Service when the result is above the alert threshold.

The Alert Service publishes **FraudAlertCreated** for the reporting or audit flow, while the frontend receives the alert through the WebSocket connection.

Events should contain an event ID, transaction ID, event type, timestamp, trace ID, and schema version. This allows events to be tracked and safely replayed. Consumers should be idempotent so that a retry does not create duplicate results or alerts.

5. Transaction flow



The first API response confirms that the transaction was accepted. The analysis result arrives shortly afterward. The Reporting Service also consumes the result and stores it so analysts can search previous transactions.

6. Frontend

The frontend uses REST for searching transactions, viewing details, and loading reports. It uses a WebSocket connection for new high-risk alerts. If the connection is lost, the frontend can use REST to load alerts it missed.

The API Gateway acts as a simple BFF for the frontend. It provides the frontend-facing endpoints without exposing the internal service structure. Analyst access should use the bank's identity provider and role-based permissions.

7. Technology choices

Area	Technology	Reason
Containers and scaling	Docker and Kubernetes	Allows services to be deployed and scaled independently.

Area	Technology	Reason
API	API Gateway	Provides authentication, routing, and rate limiting.
Events	Kafka	Supports high throughput, retention, and replay.
Transaction and reports	PostgreSQL	Provides consistent and queryable records.
Feature lookup	Redis	Provides fast access to recent customer features.
Machine learning	Python service	Has suitable libraries for fraud models.
Compliance storage	S3	Provides durable, low-cost long-term storage.
Observability	OpenTelemetry with central logs and metrics	Connects logs, metrics, and traces across services.

8. Scaling and resilience

Kafka topics are partitioned so several instances of a service can process events at the same time. The Fraud Detection Service can be scaled based on Kafka consumer lag and model processing time. The Transaction Service and API Gateway can be scaled based on request volume. Reporting can use database indexes and read replicas as the number of records grows.

When a service is scaled, its instances use the same Kafka consumer group. Kafka assigns each partition to only one instance in that group at a time, and reassigns partitions if an instance fails. A consumer commits its offset after the event has been processed successfully. If it fails before committing, the event may be read again by another instance. This is normally at-least-once delivery, so database writes use the transaction ID or event ID as a unique key to make retries safe. Kafka transactions and producer idempotence can provide exactly-once processing for Kafka-to-Kafka flows, but the database is external, so idempotent writes are still needed. This gives a reliable business result without creating duplicate fraud results or alerts.

The main failure protections are:

- Kafka uses multiple brokers and replicated topics, so one broker failure does not lose events.
- PostgreSQL uses high availability, backups, and point-in-time recovery.
- Failed event processing is retried and then sent to a dead-letter topic.
- Consumers use transaction IDs or event IDs to prevent duplicate processing.
- Redis is treated as a cache. If a feature is missing, the service uses a defined fallback and records that the feature was unavailable.
- Alerts are stored, so an analyst can recover missed alerts after a WebSocket failure.

Kafka and S3 provide two ways to recover data. A reporting database can be rebuilt by replaying events if necessary. The main single points of failure would be an unreplicated Kafka broker, database, or Redis instance, so these components should use replication and backups.

9. Observability

Each request and event should include a transaction ID and trace ID. OpenTelemetry can collect traces from the API, database calls, Kafka processing, Redis lookups, and model inference. Services should write structured logs to a central location and expose metrics.

The three main performance metrics are:

- **Latency:** time to acknowledge a transaction, produce a fraud result, and deliver an alert.
- **Throughput:** number of transactions accepted and analysed per second. The system should support the required peak of 1,000 transactions per second without an increasing Kafka backlog.
- **Availability:** percentage of time that the transaction API is available and able to accept valid requests.

Other useful metrics are Kafka consumer lag, fraud model errors, alert delivery time, and dead-letter messages.

Initial SLOs could be:

- 99% of valid transaction requests receive an acknowledgement within 200 ms;
- 99% of accepted transactions receive a fraud result within 2 seconds during normal load;
- 99% of high-risk results reach the connected analyst frontend within 1 second;
- 99.9% monthly availability for the transaction API, excluding planned maintenance.

Logs, metrics, and traces should use the same transaction ID and trace ID. This makes it possible to follow one transaction through the whole system.

10. Trade-offs

Kafka and microservices make the system scalable and replayable, but they add some complexity. Since events can be delivered more than once, services must handle duplicates safely.

The fraud result is produced after the transaction is accepted, so the system is eventually consistent. Redis improves speed but can have missing or old features, while PostgreSQL provides the more reliable record for reporting and audits.

A service mesh is not needed initially. The API Gateway, Kubernetes, and OpenTelemetry provide enough routing, scaling, and monitoring for this design.